

Bagging vs Random Forest

Contents

1. Overview	2
2. Difference 1: the base model	3
3. Difference 2: tree-level against node-level column sampling	3
3.1 Bagging: tree-level sampling	3
3.2 Random forest: node-level sampling	4
3.3 Why more randomness helps	4
4. Checking it in code	4
4.1 The bagged trees	5
4.2 The forest's trees	5
4.3 Does it pay off?	6
5. Summary	7
Sources	7
6. Key terms	7

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Overfitting ([Note 91](#)); Hyperparameter tuning ([Note 98](#)); Decision trees ([Note 100](#)); Feature importance ([Note 100](#)); Bagging ([Note 107](#)); OOB score ([Note 107](#)).
- **Leads to:** Balanced random forest ([Note 133](#)).
- **Compare with:** Bagging ([Note 107](#)); Dropout ([Note 1024](#)).

1. Overview

Key point: Two differences: bagging can use any base model while a random forest always uses decision trees; and bagging samples columns once per tree while a random forest samples them again at every node.

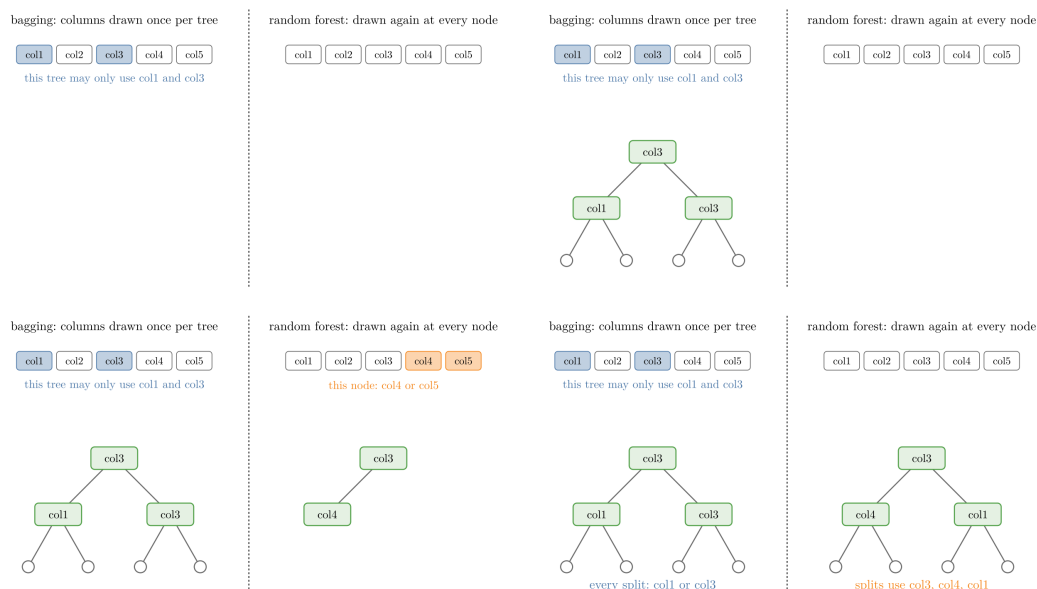


Figure 1: Column sampling with 2 of 5 columns: bagging draws the pair once and the whole tree uses it; a random forest draws a new pair at every node

A random forest is built on bagging (the [random forest introduction Note](#)), but the two are not the same, even when bagging uses decision trees. Figure 1 shows the subtle difference: **where** the columns are sampled.

This Note covers both differences and checks the second one in code. The Notebook ([notebook.ipynb](#)) runs every check.

2. Difference 1: the base model

Key point: Bagging is a general technique for any algorithm; a random forest is made of decision trees only.

In bagging, the base models can come from any algorithm, as long as they all use the same one: all decision trees, all KNN or all SVMs (the [bagging classifier Note](#), section 2.2). Decision trees, KNN and SVMs are the usual choices.

In a random forest, the base model is always a decision tree. scikit-learn's classes show this directly:

- `BaggingClassifier` has an `estimator` parameter for choosing the base model; when it is left at `None`, it uses a decision tree.
- `RandomForestClassifier` has `n_estimators`, the number of trees, but **no** parameter for choosing the base model.

Extra: Older documentation and code call this parameter `base_estimator`. It was renamed `estimator` and the old name was removed in scikit-learn 1.4 (the [bagging classifier Note](#), section 3.2).

3. Difference 2: tree-level against node-level column sampling

Key point: Bagging fixes each tree's columns before the tree is grown; a random forest picks a fresh random set of columns before every split.

So is a bagging ensemble of decision trees a random forest? **No.** The remaining difference is in how the columns are sampled (column sampling: the [random forest introduction Note](#), section 4).

Take a dataset with 5 columns, and suppose each tree may use 2 of them (`max_features=2` in both classes).

3.1 Bagging: tree-level sampling

Key point: Two columns are drawn once; every split in that tree uses one of those two.

Before tree 1 is grown, 2 of the 5 columns are drawn at random, say `col1` and `col3`. Tree 1 is then trained on those two columns only, and every split in it is on `col1` or `col3` (Figure 1, left). Tree 2 gets its own pair, say `col4` and `col5`; tree 3 `col1` and `col4`; and so on.

This is **tree-level column sampling**: the columns are decided once, before the tree is built, and no other column is ever touched by that tree.

3.2 Random forest: node-level sampling

Key point: Two columns are drawn at every node, so one tree can end up using all five.

In a random forest, the draw happens again **every time a node is about to split** (Figure 1, right):

- the root draws col2 and col3, and splits on the better of the two, col3;
- the next node draws again, col4 and col5, and splits on col4;
- the next draws col2 and col1, and splits on col1.

This is **node-level column sampling**. It is exactly the `max_features` setting of a single decision tree (the [decision tree hyperparameters Note](#), section 4.6), applied in every tree of the forest. It adds more randomness than tree-level sampling.

3.3 Why more randomness helps

Key point: More randomness makes the trees more different from each other, and an ensemble of different models performs better.

From the ensemble Notes (the [voting ensemble Note](#)): an ensemble works best when its base models are each better than chance and as different from each other as possible. Node-level sampling makes the trees more different, so a random forest usually beats a bagging ensemble of trees.

“Usually” is the right word: on some datasets bagging wins, so it is worth trying both.

Extra: Why does difference between the trees matter so much? Suppose each tree’s prediction has variance σ^2 and any two trees are correlated by ρ (0: unrelated, 1: identical).

1. **In words:** the variance of the average of n trees has a part that more trees can remove and a part, set by the correlation, that they cannot.
2. **Formula:**

$$\text{variance of the average} = \rho \sigma^2 + \frac{1 - \rho}{n} \sigma^2$$

3. **Example:** with $\sigma^2 = 1$ and $n = 100$ trees: for $\rho = 0.6$, $0.6 + 0.4/100 = 0.604$; for $\rho = 0.3$, $0.3 + 0.7/100 = 0.307$.

Adding trees only shrinks the second term. The first term stays, however many trees we add, unless the trees become less alike. Lowering ρ is exactly what node-level sampling does. (ESL §15.2, eq. 15.1)

4. Checking it in code

Key point: Every bagged tree splits on exactly 2 columns; the forest’s trees split on 2 to 5 columns, most often 4.

We use a dataset of 100 rows and 5 input columns, col1 to col5, made with `make_classification`, and give both classes `max_features=2`.

4.1 The bagged trees

Key point: The first bagged tree only ever splits on col5 and col1, the two columns it was given.

Python: Bagging with 2 columns per tree, and the columns each tree got.

```
from sklearn.ensemble import BaggingClassifier
```

```
bag = BaggingClassifier(max_features=2, random_state=42)
```

```
bag.fit(df.iloc[:, :5], df["target"])
```

```
bag.estimators_features_[0]      # array([4, 0])
```

bag.estimators_[0] is the first trained tree. estimators_features_[0] lists the columns it was given (the [bagging classifier Note](#), section 3.3): positions 4 and 0, that is col5 and col1.

When we print the first tree, every split is on feature_0 or feature_1. Careful: these are **not** col1 and col2. The tree was trained on a 2-column table, so it numbers those two columns 0 and 1 itself; estimators_features_ maps them back to col5 and col1.

Python: Printing a tree with the real column names.

```
from sklearn.tree import export_text
```

```
cols = bag.estimators_features_[0]
```

```
print(export_text(bag.estimators_[0],  
                 feature_names=list(df.columns[cols])))
```

export_text prints a tree as indented text, one line per split. feature_names replaces feature_0, feature_1, ... with names.

All 10 bagged trees use exactly 2 columns: (col1, col5), (col1, col3), (col3, col5), and so on.

4.2 The forest's trees

Key point: The first forest tree splits on col3, col5, col1 and col2: four different columns with max_features=2.

The first tree of RandomForestClassifier(max_features=2) splits first on col3, then on col5, col1 and col2. Every tree of a forest is trained on all the columns, so its column numbers are the real ones.

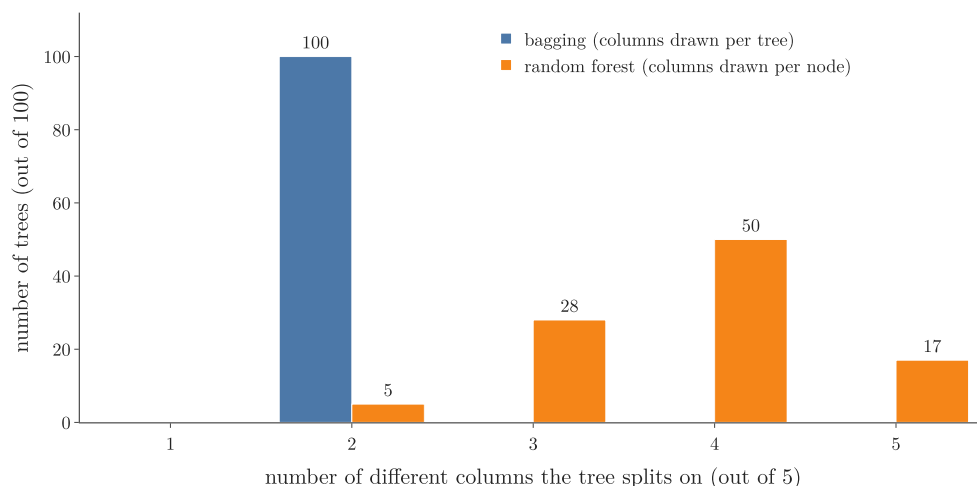


Figure 2: How many different columns each of 100 trees splits on, out of 5, with `max_features=2`

Figure 2 counts, for 100 trees of each kind, how many different columns each tree splits on:

- **bagging:** all 100 trees use exactly 2 columns;
- **random forest:** 5 trees use 2 columns, 28 use 3, 50 use 4, and 17 use all 5.

That is the proof: bagging samples columns at the tree level, a random forest at the node level.

4.3 Does it pay off?

Key point: On 2,000 rows and 20 columns, the forest scores 0.917; bagging scores 0.913 with all columns and 0.892 with 5 columns per tree.

On a larger dataset (2,000 rows, 20 columns, 5-fold cross-validation, 100 trees each):

Model	Columns	Accuracy
Bagging	all 20 per tree	0.913
Bagging	5 per tree (tree level)	0.892
Random forest	4 per node ($\sqrt{20} \approx 4$, the default)	0.917

Here the random forest wins by a small margin, and tree-level column sampling is the worst of the three.

Extra: A bagging ensemble of trees that each sample columns at every node, `BaggingClassifier(DecisionTreeClassifier(max_features="sqrt"))`, scores exactly 0.917 too. So with node-level sampling, bagged trees **are** a random forest, which is how scikit-learn defines one (scikit-learn User Guide §1.11).

5. Summary

	Bagging	Random forest
Base model	any algorithm (estimator)	decision trees only
Column sampling	once per tree (tree level)	at every node (node level)
Columns one tree can use, with <code>max_features=2</code> of 5	exactly 2	up to all 5
Randomness	less	more
On our 20-column data	0.913 (all columns)	0.917

- A bagging ensemble of decision trees is still not a random forest.
- Bagging decides each tree's columns before it is grown; a random forest re-draws them before every split.
- More randomness makes the trees less alike, which usually makes the forest better; try both.
- In a bagged tree, `feature_0`, `feature_1`, ... are positions within that tree's own columns; `estimators_features_` maps them back.

Sources

- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed. Springer.
- scikit-learn developers. User Guide, section 1.11, "Ensembles" (Random forests). <https://scikit-learn.org/stable/modules/ensemble.html>

6. Key terms

Term	Meaning
Tree-level column sampling	Drawing one random set of columns per tree, before the tree is grown; every split of that tree uses only those columns (bagging)
Node-level column sampling	Drawing a new random set of columns before every split (random forest)
<code>export_text</code>	scikit-learn function that prints a trained tree as indented text
Correlation between base models	How alike two base models' predictions are; the less alike, the more an ensemble cuts variance